最终可投递版（后端 / 架构向）

InfiniteChat 分布式即时通信系统 | [时间按实际填写] | 独立开发 / Java 后端

背景：面向即时通信与轻社交场景，针对 IM 场景中高频发送、低延迟推送、消息可靠性与在线状态一致性等问题，搭建可验证、可压测、可恢复的后端架构。

目标：构建基于 Spring Cloud + Netty + RocketMQ + Redis 的分布式消息链路，实现消息有序投递、在线精准路由、离线补偿及高并发红包处理，并通过压测量化验证性能瓶颈与故障恢复能力。

过程：

- 构建 Netty WebSocket 接入层，采用 Boss / Worker / Business 三层线程模型，实现单设备在线和跨节点精准推送。节点重启后通过指数退避 + 随机抖动的重连策略将恢复阶段失败重试从 2000 次降至 126 次。
- 实现消息主链路幂等与有序语义，使用 Redis Lua 原子完成去重与会话级序号分配，结合 RocketMQ 有序消息、异步落库和 failover 重试，保证“至少一次投递 + 消费端去重”。主导三轮 wrk2 压测与性能调优，定位了事务长占连接、伪同步 ACK、Redis 连接池不足等瓶颈，将消息发送入口从 491 RPS / P99 48s 优化至 994 RPS / P99 80ms。
- 设计离线消息热冷分层存储，推送失败消息回流 OfflineService，按 Redis ZSet（3 天热数据）+ MySQL（7 天冷数据）双写，并支持增量补拉与未读计数维护。
- 实现红包事务链路，发红包通过 MySQL 行锁 / CAS 完成余额扣减，抢红包使用 Lua 原子脚本 + RocketMQ 事务消息保证 Redis 预扣与 MySQL 落库最终一致，防止超发和重复领取。红包链路在 800 RPS 下 P99 51ms、零错误，数据一致性验证零超发、零重复领取。

结果：在 2C8G 单节点环境下，RTC 1000 长连接稳定在线，高频心跳场景无异常断连；消息发送接口在 1000 RPS 下无超时无错误；红包链路在 800 RPS 下 P99 51ms、零错误，数据一致性验证零超发、零重复领取。

技术栈：Java、Spring Boot、Spring Cloud Gateway、Netty、RocketMQ、Redis / Redisson、MySQL、MyBatis-Plus、Nacos、OpenFeign、JWT

InfiniteChat 分布式即时通信系统 | [时间按实际填写] | 独立开发 / Java 后端

背景：面向即时通信与轻社交场景，围绕高频发送、低延迟推送、消息可靠性与在线状态一致性等核心问题，设计可验证、可压测、可恢复的分布式 IM 后端架构。

目标：构建基于 Spring Cloud + Netty + RocketMQ + Redis 的分布式消息链路，实现会话级有序投递、在线精准路由、离线补偿及高并发红包处理，并通过压测量化验证系统瓶颈与故障恢复能力。

过程：

构建 Netty WebSocket 接入层，采用 Boss / Worker / Business 三层线程模型，实现单设备在线与跨节点精准推送；节点重启后通过指数退避 + 随机抖动重连，将恢复阶段失败重试从 2000 次降至 126 次。

实现消息主链路幂等与会话级有序语义，使用 Redis Lua 原子完成去重与会话级序号分配，结合 RocketMQ 顺序分片投递、异步落库与 failover 重试，保证“至少一次投递 + 消费端去重”；主导 3

轮 wrk2 压测与调优，定位事务长占连接、伪同步 ACK、Redis 连接池不足等瓶颈，将发送入口从 491 RPS / P99 48s 优化至 994 RPS / P99 80ms。

设计离线消息热冷分层存储，推送失败消息回流 OfflineService，按 Redis ZSet（3 天热数据）+ MySQL（7 天冷数据）双写，并支持基于 `sessionId + seq` 的增量补拉与未读计数维护。

实现红包事务链路，发红包通过 MySQL 行锁 / CAS 完成余额扣减，抢红包使用 Lua 原子脚本 + RocketMQ 事务消息保证 Redis 预扣与 MySQL 落库最终一致，防止超发和重复领取；红包接口在 800 RPS 下 P99 51ms、零错误，一致性验证零超发、零重复领取。

结果：在 2C8G 单节点环境下，RTC 1000 长连接稳定在线，高频心跳场景无异常断连，节点重启后约 8s 恢复至 100% 活跃连接；消息发送入口在 1000 RPS 目标压测下实测 994 RPS、P99 80ms、零超时零错误；红包接口在 800 RPS 下 P99 51ms、零错误，一致性验证零超发、零重复领取。

技术栈：Java、Spring Boot、Spring Cloud Gateway、Netty、RocketMQ、Redis / Redisson、MySQL、MyBatis-Plus、Nacos、OpenFeign、JWT

Netty接入层（简历版）

Netty 长连接接入层：基于 Netty 自建 WebSocket 长连接服务，采用 Boss/Worker/Business 三层线程模型隔离 IO 与业务，并维护 Redis 全局在线路由，感知断连后主动双清理；在 2C8G 单节点环境下完成 RTC 专项压测，1000 长连接在高频心跳场景下稳定在线，节点重启后通过指数退避 + 随机抖动的重连策略将恢复阶段失败重试从 2000 次降至 126 次，并在约 8s 内恢复至 100% 活跃连接。

1. 一句话层

这块我主要做的是 RTC 长连接接入层，基于 Netty 搭了 WebSocket 服务，把握手鉴权、在线路由和断线重连这几个核心问题串起来了，后面我还在 2C8G 单节点环境下做了专项压测。

2. 方案层

为什么这里选 Netty 自建 WebSocket 长连接接入层，而不是直接用 Spring WebSocket?

A：Spring WebSocket 更适合快速实现业务层的 WebSocket 功能，但我这里做的是独立的 RTC 长连接接入层，不只是收发消息，还要自己控制连接建立、握手鉴权、心跳保活、在线路由、断连清理、单设备踢人和优雅停机这些底层能力。相比之下，Netty 对线程模型、Pipeline 和连接生命周期的控制更细，所以我选了 Netty。性能收益是附带结果，但不是我最核心的选型原因。

为什么要拆 Boss / Worker / Business 线程模型，而不是只用 Netty 默认的两组线程?

A：我这里把线程模型拆成 Boss、Worker 和 Business 三层。Boss 线程只负责接收连接，数量是 1；Worker 线程负责网络 IO 和编解码，数量按 CPU 核心数配置；Business 线程负责业务处理，数量是 CPU 核心数的 2 倍。这样做的核心目的，是把网络读写和业务逻辑隔离开，避免业务处理占住 Worker 线程，影响后续连接的读写。因为在我的 RTC 模块里，除了基础的心跳和协议处理外，还会涉及 JWT 鉴权、在线路由维护、断连清理和 Redis 交互，如果这些逻辑直接压在 Worker 上，在连接数上来或者节点重启集中重连时，容易放大网络处理延迟。所以我单独拆了 Business 线程池，让 IO 尽量保持轻量、业务处理异步下沉。

线程数是怎么决定的?

A：我这里不是把线程数当成拍脑袋定出来的最优值，而是先按模块职责和 Netty 的常见工程经验给了一个初始配置，再通过压测去验证它在当前环境下是否够用。具体来说，Boss 线程只负责监听和接

收新连接，不参与后续业务处理，所以先配成 1；Worker 线程负责连接上的网络 IO、编解码和 Pipeline 里的轻量处理，这部分本质上是 NIO 事件循环，所以我先按 CPU 核数来配；Business 线程负责真正的业务逻辑处理，比如握手后的业务处理、在线路由维护、断连清理以及 Redis 交互，这部分不能直接压在 Worker 上，否则业务逻辑一旦变重，就会反过来影响网络读写，所以我单独拆出来，并先给了 CPU * 2 的线程数作为起点。这里的 CPU 和 CPU * 2 不是说我已经证明它是全局最优，而是一个比较常见、合理的初始值。后面我再结合 RTC 专项压测去验证，比如观察 1000 长连接、高频心跳和节点重启恢复场景下的连接稳定性、错误数、GC 和恢复时间。至少从目前测试结果看，这套线程配置在当前 2C8G 单节点环境下没有暴露出明显瓶颈，所以我认为它对当前规模是够用的。

为什么把 JWT 鉴权放在握手阶段，而不是连接建立以后再做业务鉴权？

A：我把 JWT 鉴权放在握手阶段，核心是为了尽早失败，避免非法连接先完成 WebSocket 升级再进入后续处理流程。因为我的 RTC 模块不只是收发消息，后面还要做用户身份绑定、在线路由注册、心跳保活和断连清理，这些逻辑都要求连接一开始就是一个合法用户连接。所以我选择在握手阶段就完成 token 校验和用户身份确认，把未认证请求拦在接入层外面。

为什么在线路由要做本地会话映射 + Redis 全局路由，而不是只保存本地 Channel？

A：因为只保存本地 Channel 只能解决当前节点怎么给本机用户发消息，解决不了集群里某个用户到底连在哪个节点。如果没有全局在线路由，跨节点推送时就只能让每个 RTC 节点都尝试投送一遍，代价太大。所以我做成两层：本地会话映射负责节点内快速找到 Channel，Redis 全局路由负责在集群范围内定位用户所在节点，这样才能做到跨节点的精准投送。

为什么 Redis 全局在线路由要带 TTL，而且还要在断连时主动清理？

A：因为 TTL 和主动清理解决的不是同一个问题，所以两者都要有。主动清理负责正常场景下的及时收敛，TTL 负责异常场景下的兜底自愈。具体来说，用户正常断连时，服务端是能感知到 channelInactive 的，这时候就应该立刻删除本地会话和 Redis 全局路由。这样做的好处是在线状态收敛快，后续消息不会继续投到已经断开的节点上。如果只靠 TTL，不主动删路由，那即使用户已经离线，Redis 里的路由也还会继续保留一段时间，这段时间内全局在线状态就是滞后的。但如果只做主动清理、不加 TTL，也有问题。因为在节点 crash、进程被强杀、机器断电这类异常场景下，服务端可能根本来不及走断连回调，Redis 路由就会永久残留，形成脏数据。这样后续系统还会继续把消息路由到一个已经失效的节点。所以 TTL 的作用不是表达精确在线时间，而是保证异常情况下路由最终一定会自己过期，不会无限脏下去。所以我这里的设计是：主动清理为主，TTL 兜底为辅。正常情况下优先走断连删除；异常情况下如果来不及删，就靠 TTL 自愈。这样既保证在线状态尽量实时，又避免脏路由永久残留。

为什么 RTC 节点重启后会出现重连风暴，你是怎么优化的？

A：因为节点重启时，原来挂在这个节点上的长连接会在很短时间内一起断开。如果客户端都按固定延迟立即重连，就会出现大量连接在同一时刻重新握手，形成瞬时洪峰，这就是重连风暴。它的问题不在于最终能不能连回来，而在于恢复窗口里会产生很多无效重试，放大握手、鉴权和网络抖动成本。我在专项压测里就观察到了这个问题：优化前 1000 连接在节点重启后为了恢复满连，一共发起了 3000 次重连尝试，其中 2000 次失败；后面我把重连策略改成了指数退避 + 随机抖动，让重连请求错峰发起，避免所有连接同一时刻撞回来。优化后同样是 1000 连接重启恢复场景，总尝试次数降到 1126，失败次数降到 126，并且仍然能在约 8s 内恢复到 100% 活跃连接。更准确地说，这个优化我

目前是先在压测脚本这个模拟客户端里验证的；如果后续做真实客户端，重连退避、抖动和错误分类逻辑就应该下沉到客户端连接管理层。

为什么单节点只能做到快恢复，做不到真正无感？

A：因为单节点重启时，在那段时间里服务本身就是不可用的，客户端没有别的节点可连，所以连接一定会中断。也就是说，单节点能做的是尽快恢复，但做不到真正意义上的无感。如果想接近无感，至少要两台 RTC 节点：一台节点下线或重启时，客户端还能被重新调度到另一台可用节点；再配合消息补拉机制，比如按 seq 或 lastAckSeq 补同步缺失消息，这样连接层虽然短暂中断，但业务层体验可以基本无感。所以我现在这套方案更准确的定位是单节点快恢复方案，重点在于降低恢复窗口内的无效重试和抖动；如果要进一步升级到接近无感，就需要多节点接入、节点摘流和重连后消息补偿。

3. 细节层

本地会话表中存放的是什么？

A：本地会话表里存的是 Netty 的 Channel 对象，本质上就是服务器侧对一条客户端连接的抽象。它不是我自己定义的简单实体，而是 Netty 提供的连接对象，后续可以用它判断连接状态、绑定用户属性、执行 writeAndFlush 推送消息，或者在单设备踢人时主动关闭连接。所以本地会话表维护的是 userId -> Channel，用来支持当前节点内的快速定位和直接推送。

Channel 里为什么还要再绑定 userId 属性，明明本地 Map 已经有 userId -> Channel 了？

A：因为本地会话表解决的是通过 userId 找 Channel，但很多运行时场景拿到的起点其实是 Channel 本身，比如断连回调、心跳处理、消息处理。这时候如果提前在 Channel 上绑定了 userId，就可以直接从连接反查出用户身份，方便做断连清理、路由删除和业务处理；否则就只能遍历本地会话表去反查，代价更高。所以我这里实际上做的是一组双向索引。

详细说一下心跳、空闲检测和 TTL 的时间是怎么设置的。

A：我这里把保活频率和死亡判定分开设计了。客户端 30 秒发一次心跳，是为了维持连接活跃和刷新路由 TTL；服务端并不是丢一次心跳就断开，而是采用 5 分钟读空闲、连续 3 次才关闭的保守策略，也就是大约 15 分钟才会判定连接失活。与此同时，我又把 Redis 路由 TTL 设为 20 分钟，让正常断连优先走主动清理，异常场景再由 TTL 兜底。

那这个 15 分钟是不是太长了？

A：是的，如果目标是更实时地反映在线状态，这个值偏保守。当前配置更偏向稳定性优先，适合项目阶段先保证连接不被轻易误踢。如果后续要往生产化优化，我会把空闲剔除时间收紧到心跳周期的 3 到 6 倍，比如 90 到 180 秒，并继续观察误踢率和重连成本。

那为什么你项目里最后还是接受了这个偏保守的 15 分钟配置？

A：因为我当前阶段更看重连接稳定性，误踢的代价比晚一点判掉线更高，所以我采用了一个偏保守的空闲关闭配置。客户端 30 秒发一次心跳，服务端不是丢一两次就断，而是 5 分钟读空闲、连续 3 次才关闭，大约 15 分钟判定失活。这样做可以减少网络抖动导致的误踢。同时我又配合了主动断连清理和 Redis TTL 兜底，所以这个值在当前项目阶段是可接受的。后续如果往生产化推进，我会把它进一步收紧到 90 到 180 秒，再结合误踢率和重连成本继续调优。

详细说一下压测是怎么做的。

A：我这轮压测重点不是追求单节点绝对上限，而是验证 RTC 接入层在长连接、高频心跳和节点重启场景下的稳定性。我先用 ws_load_test.js 模拟客户端连接和心跳，逐步验证 10、20、50、500、

1000 长连接的稳定在线情况；后面再把心跳频率提高，验证高频保活下的表现；最后做节点重启恢复测试，观察 active、opened、closed、errors、reconnectOk 这些指标怎么变化。更准确地说，这轮压测验证的是 RTC 接入层的稳定性和恢复能力，而不是完整生产环境下的极限容量上限。

为什么你说这轮压测有价值，但还不能说测出了 RTC 的绝对上限？

A：因为我现在测到的是 2C8G 单节点、1000 测试用户、当前心跳模型下的 RTC 接入层表现，它能证明这套接入层在当前规模下是稳定的，也能证明节点重启后的恢复能力和重连策略优化有效；但它不能直接推出更大规模、更多节点、真实消息推送流量下的系统绝对上限。真正要做容量上限评估，还需要更大规模用户数据、更长时间 soak test、多节点环境以及更完整的消息推送压力一起验证。

为什么重连策略里要加 jitter，而不只是指数退避？

A：jitter 就是随机抖动，指的是在原本的重连等待时间上再加一个随机延迟。因为如果只有固定延迟或纯指数退避，所有客户端还是会按同样节奏一起重连，容易形成新的重连风暴；加了 jitter 以后，重连请求会被打散，服务恢复过程会更平滑。

active / attempted / opened / closed / errors / reconnectOk 这些压测指标分别怎么解释？

A：这些指标里，active 表示当前活跃连接数，attempted 表示累计连接尝试次数，opened 表示累计成功打开次数，closed 表示累计关闭事件次数，errors 表示客户端侧错误次数，reconnectOk 表示重连成功次数占总重连尝试次数。要注意的是，除了 active 以外，大多数都是累计值，所以像重启恢复场景里 opened 会大于目标连接数，这是因为初始建连和后续重连成功都会累计进去。

为什么你的重启恢复测试里 closed=1126，而不是 1000？

A：因为 closed 统计的是累计关闭事件次数，不只是最初那 1000 个在线连接在节点重启时断开的次数。我的那轮测试里，前 1000 次 close 来自节点重启导致的原始连接断开；后面服务恢复前又有 126 次重连尝试失败，这些失败连接对象同样会触发 close，所以最终 closed 变成了 1126。

那为什么 errors=126 和 closed 多出来的 126 基本能对上，但又不能简单理解成完全一一对应？

A：因为 error 和 close 统计的是两个不同维度的事件。error 表示连接过程中发生了错误，close 表示连接对象最终关闭，所以一次失败重连可能先触发 error，再触发 close；但像节点重启导致的原始在线连接断开，很多时候只会记成 close(1006)，不一定先有 error。所以我那轮测试里，前 1000 次 close 来自原始连接断开，后面多出来的 126 次 close 大体对应失败重连，而 errors=126 说明这些失败重连里发生了明确的连接错误。它们这轮看起来基本对上，但不能机械理解成 error 和 close 永远一一对应。

压测部分：

这块压测我没有一上来追求单节点绝对上限，而是按“接入层稳定性 -> 高频保活 -> 节点重启恢复”三个阶段做的。环境上我是把中间件放在 2C4G 机器，业务和 RTC 放在 2C8G 单节点机器，压测时只直连 RTC 的 WebSocket 地址，不走 Gateway，这样能把结果尽量聚焦到 Netty 长连接接入层本身。

第一步我先做环境可用性检查，确认 Redis、MySQL、Nacos、RocketMQ 都正常；然后启动 AuthenticationService 和 RealTimeCommunication，通过批量登录脚本生成测试 token。压测脚本是我自己写的 ws_load_test.js，它会模拟客户端建连、定时发心跳，并输出 active / attempted / opened / closed / errors / reconnectOk 这些指标。其中 active 表示当前活跃连接数，attempted/opened/closed/errors 是累计过程指标，reconnectOk 专门看重连恢复效果。

第二步我做了阶梯式稳定性验证。先用 10、20 连接做冒烟，确认握手成功、连接稳定、没有异常断开；之后再逐步升到 50、500、1000 连接。结果是 1000 长连接在单节点下可以稳定在线，opened=1000、closed=0、errors=0、unexpected=0，说明当前规模下 RTC 接入层是稳定的。再往后我没有继续盲目堆连接数，而是把心跳频率从 30s 提高到 10s，验证高频保活场景。这个过程中连接仍然稳定，JVM 侧我用 jstat -gcutil 观察到只发生了正常的 Young GC，没有新增 Full GC，老年代占用稳定在大约 15%，所以我判断当前线程模型和连接处理链路在 2C8G 单节点下还没有明显 JVM 层瓶颈。

第三步我做了节点重启恢复测试。最开始用固定延迟重连，结果 1000 连接在节点重启后虽然最终能恢复满连，但恢复窗口里总共发起了 3000 次重连尝试，其中 2000 次失败，典型就是重连风暴。后来我把模拟客户端的重连策略改成“指数退避 + 随机抖动”，也就是在原有重连等待时间上继续加随机延迟，把重连请求错峰。优化后，同样是 1000 连接、节点重启场景，总重连尝试降到 1126，失败次数降到 126，并且仍然能在大约 8s 内恢复到 100% 活跃连接。这个结果说明我虽然做的是单节点方案，做不到真正无感，但已经把“快恢复”这件事做得比较平滑了。

所以这条项目点我最后想强调的不是“我测到了某个很夸张的上限”，而是：我把 RTC 接入层的稳定性、高频保活和故障恢复链路完整跑通了，还通过压测定位到了重连风暴问题，并给出了优化前后的量化结果。

命令、指标和结果

消息主链路（简历版）

消息发送主链路：以 Redis Lua 脚本将去重判定与会话级递增合并为单次原子操作，消除幂等判断与序号分配之间的竞态；校验路径全走 Redis Cache（用户/好友/群成员均采用 Cache-Aside + 随机 TTL 防雪崩），MQ 投递通过专用 `mqSendExecutor` 线程池异步提交，Tomcat 线程立即返回 `messageId + seq`，不等 broker 确认；`MessageStoreListener` 并发消费 `IM_MSG_STORE` 落库后，以 `multiGet` 批量查询接收方在线路由，将 `GatewayPushPacket` 按节点定向转发至各 RTC 节点或离线队列，MQ 发送失败写入 `im_msg_failover` 并由定时任务加 Redisson 分布式锁重投；经 wrk2 三轮压测调优，依次定位并修复 `@Transactional` 长占连接、假同步 ACK 阻塞（`future.get` 等 broker 确认）、Redis 连接池不足与重复调用等瓶颈，发送入口在 2C8G 单机目标 1000 RPS 下从 491 RPS/P99 48s 优化至 994 RPS/P99 80ms、零超时零错误。

离线消息与补拉：在线推送失败时消息回流 `IM_CHAT:OFFLINE` 队列，`OfflineMessageListener` 并发消费后对每位目标用户双写：以 `im:offline:{userId}:{sessionId}` 为键存入 Redis ZSet（score=seq，TTL 3 天热数据）并落 MySQL（expire_at 字段标 7 天冷数据，先 `countByReceiverAndMessageId` 防重复写入），同步通过 Redis Hash（`im:unread:{userId}`，field=sessionId）累加各会话未读数，读时 Cache-Aside 回填 MySQL；补拉接口 `POST /api/offline/pull` 支持单会话（`sessionId + lastSeq`）与全量会话（`Map<sessionId, seq>`）两种模式，优先以 `minHotSeq`（ZSet 最小 score）判断冷热边界，热区内走 ZSet `ZRANGEBYSCORE`（score > lastSeq 开区间，拉取后即从 ZSet 删除），超出热区降级至 MySQL；Redis 计数器限流（10 次/秒/用户）防重连风暴冲击接口；每日凌晨 2 点定时任务以 Redis SETNX 单实例加锁，分批清理过期（> 7 天）与已读（status=1, > 3 天）离线记录。

for f in /tmp/messaging.1.1775476946.jstack /tmp/messaging.1.1775476975.jstack; do echo "===== $f =====" grep "java.lang.Thread.State" "$f" | sort | uniq -c done

java.lang.Thread.State 这一步是在把每个线程当前状态提取出来。

| sort 把刚才提取出的状态行排序，让相同状态的行排在一起 | uniq -c 对排序后的相同行做计数，告诉你每种线程状态出现了多少次 先看 JVM 里线程整体在干嘛，是在跑，还是在等，还是在抢锁。

统计两份 jstack 里，各种线程状态分别有多少。

for f in /tmp/messaging.1.1775476946.jstack /tmp/messaging.1.1775476975.jstack; do echo "===== $f =====" grep '^"' "$f" | sed -E 's/^"([^"]+)".*/\1/' | sed -E 's/-[0-9]+$//' | sort | uniq -c | sort -nr | head -20 done

grep '^"' "$f" | sed -E 's/^"([^"]+)".*/\1/' | sed -E 's/-[0-9]+$//' | sort | uniq -c | sort -nr | head -20

"http-nio-8084-exec-12" #123 prio=5 os_prio=0 tid=... "ConsumeMessageThread_1" #234 prio=5 os_prio=0 tid=... "VM Thread" os_prio=0 tid=... 所以这一步是在提取：每个线程的“名字行”

| sed -E 's/^"([^"]+)".*/\1/' 用正则把线程头行里线程名字本身提出来去掉后面的 #123 prio=... 这些信息

| sort -nr 再按数量倒序排序：最多的线程组排最前面 这样你能很快看出：当前 JVM 里主要是哪些线程占多数

1000 QPS 出现明显超时和多秒级尾延迟时，主要症状是大量 http-nio-8084-exec 请求线程被占住，而不是 JVM 里大量线程都在 RUNNABLE 疯跑，所以这更像等待/阻塞型瓶颈，而不是纯 CPU 计算瓶颈。

http-nio-8084-exec-123 8084 端口上，一个正在处理请求的 Tomcat 工作线程。

入口发送接口在高压下首先暴露出来的是“请求线程被数据库事务相关操作卡住”，而且这种卡住表现为：一部分线程慢在 MySQL commit，另一部分线程堵在连接池借连接。

我先做无偏排查，发现 1000 QPS 时大量 http-nio 请求线程并不是纯 CPU 跑满，而是卡在数据库事务相关路径上：一部分线程在 Hikari 连接池等待连接，另一部分线程在 MySQL commit 阶段等待返回。这说明高压下入口线程首先被数据库事务链路占住。后续再结合代码设计看，发送接口本身还叠加了假同步 ACK，消费者侧还有顺序消费内同步下游发送，这些都会进一步放大尾延迟。

我当时先用 wrk2 做阶梯压测，发现 /api/message/send 在 1000 QPS 左右出现明显拐点，实际吞吐只能到大约 491 RPS，P99 到了 48 秒，还出现了 1764 次超时。然后我先在压测高峰抓了两次 jstack，先看线程状态分布和线程组分布。结果发现大量线程是 waiting/parking，不是纯 CPU 打满；同时最集中的线程组是 200 个 http-nio-8084-exec，说明主要是 HTTP 请求线程被占住了。 接着我再抽样看这些请求线程的调用栈，发现大量线程都卡在 HikariPool.getConnection，少量线程卡在 MySQL commit 的 socket 读上。结合代码一看，sendMessage 这个接口方法本身挂了事务，事务一开始就要借数据库连接，但这个方法里面又不只是简单写库，还做了发送校验、缓存回源、组包、发 MQ，而且旧版实现里还会 future.get(3s) 等 MQ 回调再 ACK。这样就把数据库连接占用时间拉长了。与此同时，MQ 消费端落库也在同一个 Messaging 进程里，共用同一个 Hikari 连接池，而连接池最大只有 30。最后就形成了一个链路：少量线程 commit 变慢，连接释放变慢，后面的请求线程大量堵在连接池门口，HTTP 工作线程被占满，最终表现出来就是接口排队、尾延迟飙升和超时。

我先去掉了发送入口外层事务，第二轮 1000 QPS 回升到约 701 RPS，P99 降到 30.0s，timeout 降到 43，说明第一刀是有效的。 • 然后再抓新一轮 jstack。J1 里有类加载和 fastjson 预热竞争，但 J2

里这些都消失了；J2 前几个 http-nio 线程统一停在 CompletableFuture.get -> sendMessage:169，这才把稳态主瓶颈定位到“假同步 ACK”。

"我这个项目的消息发送接口我做了三轮压测和优化。前两轮我用 wrk2 + jstack 的组合，定位了事务占连接、假同步 ACK 阻塞这两个问题，最终把接口从 491 RPS、P99 48s 优化到了 1200 QPS、P99 72ms。" "第三轮我把目标压力提高到 1000 RPS、300 并发连接，P50 还是 106ms，但 P99 飙到了 2s。这种双峰分布的特征和前两轮不一样——它不是'整体慢'，而是'大多数快、少数极慢'，这个模式通常指向某个有限资源被偶发争抢。" "我用前两轮同样的方法抓 jstack，看到大量 http-nio 线程卡在 Lettuce 连接池借连接，不是在计算也不是在等 MySQL。然后我用 Little's Law 算了一下：1000 RPS × 0.106s = 大约 106 个请求同时在途，而 Redis 连接池最多只配了 32 个连接，明显不够。" "同时我注意到 P99 刚好卡在 2001ms，和代码里 asyncSendOrderly 的 2000ms 超时对上了，说明 RocketMQ producer 内部队列在高压下也存在阻塞。" "所以我做了两件事：把 Redis max-active 从 32 调到 128，消除连接池瓶颈；同时把群聊场景里重复调用两次 getGroupMemberIdsWithCache 的代码合并成一次，减少每个请求的 Redis 调用数。"

消息发送接口三轮压测与调优（完整过程记录）

一、环境说明与部署方式

服务器配置：

中间件机器（2C4G）：部署 Redis、MySQL、RocketMQ、Nacos

业务机器（2C8G）：部署 Gateway（端口 10010）、Messaging（端口 8084）、RealTimeCommunication

部署方式： 每次代码改完后在本地用 Maven 打包成 fat jar，再 scp 上传到服务器：

```
# 本地打包
mvn clean package -DskipTests -pl Messaging

# 上传到服务器
scp target/Messaging-1.0-SNAPSHOT.jar yutao@101.37.65.249:/home/yutao/apps/

# 服务器上停掉旧进程，启动新版本
kill $(cat /home/yutao/apps/messaging.pid)
nohup java -jar /home/yutao/apps/Messaging-1.0-SNAPSHOT.jar \
  > /home/yutao/logs/Messaging.log 2>&1 &
echo $! > /home/yutao/apps/messaging.pid
```

压测工具： wrk2，一个基于恒速模型的 HTTP 压测工具。与普通 wrk 的区别在于：wrk 是尽力发，实际 QPS 由服务器性能决定；wrk2 用 `-R` 指定目标发送速率，服务器跟不上时超时数会上升，更接近真实用户流量模型，也能用 HDR histogram 准确记录尾延迟。

压测命令：

```
/home/yutao/wrk2/wrk \
  -t8 \          # 8 个 wrk worker 线程，用来发请求
  -c300 \        # 保持 300 个并发 HTTP 长连接
  -d90s \        # 压测持续 90 秒
  -R1000 \       # 目标发送速率 1000 RPS（恒速）
  -L \           # 开启 HDR Latency histogram，输出完整百分位分布
  -s /home/yutao/loadtest/send_msg.lua \  # Lua 脚本，负责构造请求
  http://101.37.65.249:10010/api/message/send
```

Lua 脚本功能： 预加载 2000 个测试用户的 userId 和 JWT token，每次请求随机选一对 sender/receiver，构造合法的 POST body 和 Authorization header，让服务器认为是真实用户在发消息。

核心指标含义：

| 指标 | 含义 | |------|------| | Requests/sec | 实际吞吐量，低于 -R1000 说明服务器跟不上 | | P50 | 中位数延迟，代表正常请求速度 | | P99 | 99 分位延迟，代表尾部慢请求 | | P999 | 99.9 分位延迟，极端慢请求 | | timeout errors | wrk2 默认 2s 超时，超过即计入，代表服务器完全处理不过来的请求 | | non-2xx responses | HTTP 非成功响应，代表业务报错 |

二、第一轮压测：定位数据库事务瓶颈

测试端点： `http://101.37.65.249:10010/api/message/send`（经过 Gateway，端口 10010）

初始结果：

```
Requests/sec:  491.23
Latency Distribution:
  50.000%   203.45ms
  99.000%    48.12s
timeout errors: 1764
non-2xx responses: 0
```

实际吞吐只有 491 RPS，P99 达到 48 秒，1764 次超时。

排查过程：

首先确认问题类型。P99 = 48s 是整体慢，不是双峰，说明瓶颈不是偶发资源争抢，而是请求线程被系统性地阻塞住了。

在压测高峰期抓两次 jstack 快照：

```
# 获取 Messaging 进程 PID
ps aux | grep Messaging | grep -v grep

# 抓 jstack 快照（间隔约 5 秒抓两次，排除瞬时噪音）
```

```
jstack <pid> > /tmp/messaging.j1.jstack
sleep 5
jstack <pid> > /tmp/messaging.j2.jstack
```

第一步：统计线程状态分布

```
for f in /tmp/messaging.j1.jstack /tmp/messaging.j2.jstack; do
  echo "===== $f ====="
  grep "java.lang.Thread.State" "$f" | sort | uniq -c
done
```

含义：提取每个线程的当前状态（RUNNABLE / WAITING / TIMED_WAITING / BLOCKED），按类型统计数量。如果大量线程是 WAITING/TIMED_WAITING 而不是 RUNNABLE，说明是等待型瓶颈（等锁、等 I/O、等连接池），不是 CPU 打满型瓶颈。

结果：绝大多数 http-nio-8084-exec 线程都是 WAITING，只有少量是 RUNNABLE，确认是阻塞型瓶颈。

第二步：统计线程组分布

```
for f in /tmp/messaging.j1.jstack /tmp/messaging.j2.jstack; do
  echo "===== $f ====="
  grep '^"' "$f" \
    | sed -E 's/^"([^"]+)".*/\1/' \
    | sed -E 's/-[0-9]+$//' \
    | sort | uniq -c | sort -nr | head -20
done
```

含义：提取所有线程名，把末尾的编号去掉（`-12`、`-34` 这类），再按线程组名统计数量，找出数量最多的线程组。结果显示 http-nio-8084-exec 线程组有约 200 个，说明 Tomcat 工作线程全部被占住了。

第三步：查看具体调用栈

在 jstack 文件中手动抽样查看 http-nio-8084-exec 线程的调用栈，发现大量线程卡在两个位置：

`HikariPool.getConnection`：在连接池门口等待借连接

`SocketInputStream.socketRead0`：在等待 MySQL commit 的返回

根因分析：

`sendMessage` 方法头上挂了 `@Transactional` 注解，方法一进入就借一个 MySQL 连接并开启事务。但方法体内不只是简单写库，还做了用户缓存查询、好友关系校验、消息组包、发 MQ。与此同时，MQ 消费端落库也在同一个 Messaging 进程里，共用同一个 HikariCP 连接池（`maximum-pool-size: 30`）。

链路：少量线程 MySQL commit 变慢 → 连接持有时间拉长 → 连接池中连接被耗尽 → 后续请求线程全部阻塞在借连接 → Tomcat 工作线程全部被占满 → 接口排队、超时。

优化： 去掉 `sendMessage` 方法上的 `@Transactional` 注解。发消息本身不需要事务，`seq` 生成通过 Redis Lua 脚本保证原子性，落库由 MQ Consumer 异步完成。

第一轮结果对比：

| 指标 | 优化前 | 优化后 | |------|--------|--------| | 实际 RPS | 491 | 701 | | P50 | 203ms | 98ms | | P99 | 48.12s | 30.04s | | timeout | 1764 | 43 |

P99 从 48s 降到 30s，超时从 1764 降到 43，吞吐从 491 升到 701 RPS，说明事务是第一个主瓶颈，去掉以后明显改善。但 P99 仍然在 30s，说明还有别的瓶颈。

三、第二轮压测：定位假同步 ACK 瓶颈

起点： 第一轮优化后 701 RPS、P99=30s

对第一轮修改后的版本再次抓 jstack，这次 J1 里有类加载和 fastjson 预热竞争的噪音，但在 J2（稳态）里，所有 http-nio 请求线程都统一停在同一个位置：

```
java.lang.Thread.State: TIMED_WAITING (on object monitor)
  at java.util.concurrent.CompletableFuture.get(CompletableFuture.java:1953)
  at
com.wangyutao.messaging.service.impl.MessageServiceImpl.sendMessage(MessageServiceImp
l.java:169)
```

根因： 旧版实现在 `sendMessage` 里调用了 `asyncSendOrderly` 之后紧跟着 `future.get(3000ms)` 等待 MQ broker 确认 ACK，才给客户端返回。看起来是"异步发送"，实际上是"假同步"——Tomcat 线程还是被阻塞了 3 秒，等待 MQ 回包。

优化： 去掉 `future.get` 等待，改为真正的异步回调。MQ 发送结果通过 `SendCallback.onException` 回调处理：成功不做任何阻塞操作，失败则写入 `im_msg_failover` 表，由定时任务结合分布式锁重投。接口直接返回 `messageId + seq`，不等待 MQ broker 确认。

第二轮结果对比：

| 指标 | 第一轮后 | 第二轮后 | |------|----------|----------| | 实际 RPS | 701 | 920 | | P50 | 98ms | 85ms | | P99 | 30.04s | 950ms | | timeout | 43 | 0 |

吞吐从 701 升到 920 RPS，P99 从 30s 降到 950ms，超时归零。假同步 ACK 是第二个主瓶颈，去掉之后 Tomcat 线程不再被 MQ 等待占住。但 P99 仍然接近 1s，说明在高压下还有偶发的慢路径没有消除。

四、第三轮压测：定位 Redis 连接池不足与 MQ 发送阻塞 Tomcat

新增直连测试： 这轮额外增加了对 Messaging 服务 8084 端口的直连测试（不经过 Gateway），目的是排除 Gateway 自身的 JWT 验签和路由代理开销对结果的干扰，让压测结果更纯粹地反映 Messaging 服务本身的性能。

两个测试端口的区别：

10010（Gateway）：请求经过 Gateway 做 JWT 验签、路由转发，再打到 Messaging；Gateway 有自己的连接池，对极端尾延迟有一定缓冲作用

8084（直连）：请求直接到 Messaging 服务，没有中间层，压力更集中，能暴露 Messaging 侧真实瓶颈

直连 8084 初始结果（第三轮优化前）：

```
Requests/sec:  518.41
Latency Distribution:
  50.000%    61.23ms
  99.000%  1957.12ms
  99.900%  2134.56ms
timeout errors: 52
non-2xx responses: 0
```

P50 只有 61ms（快路径正常），但 P99 接近 2s，出现 52 次超时。这是典型的双峰分布：大多数请求很快，少数请求极慢。双峰模式通常指向某个有限资源被偶发争抢，而不是整体慢。

排查过程：

定位 Redis 连接池不足（Little's Law）：

用 jstack 抓栈，发现大量 http-nio 线程卡在 Lettuce（Redis 客户端）的连接池等待处，不是在计算，也不是在等 MySQL。

用 Little's Law 验证：

```
同时在途请求数 N = 吞吐量 λ × 平均响应时间 W
N = 1000 RPS × 0.061s（P50 对应的稳态延迟）≈ 61 个请求同时在途
```

而 `application.yml` 中 Redis Lettuce 连接池配置：

```yaml
spring:
  redis:
    lettuce:
      pool:
        max-active: 32   # 最大连接数
```

61 个并发请求，池子只有 32 个连接，必然有 ~30 个请求在门口等连接。尾延迟飙升的原因找到了。

定位重复 Redis 调用：

代码审查发现群聊场景下 `sendMessage` 在同一次请求里调用了两次 `getGroupMemberIdsWithCache`：一次在 `validateSenderAndReceiver` 内部，一次在组装 `targetUserIds` 时。每次调用都要从 Redis 取一次群成员列表，1000 QPS 下相当于凭空多了 1000 次 Redis 请求。

定位 asyncSendOrderly 仍在阻塞 Tomcat 线程：

P99 刚好卡在约 2000ms，和代码里 `asyncSendOrderly(..., 2000L)` 的 timeout 参数完全对上。虽然方法名叫"async"，但 RocketMQ producer 有内部 sendQueue，当 broker 写入压力大、producer 内部队列满时，`asyncSendOrderly` 会阻塞调用线程最多 timeout 毫秒等待队列有空位。直连 300 个连接下 Tomcat 线程频繁触发这个阻塞，就产生了 2s 尾延迟。

三项优化：

1. 扩容 Redis Lettuce 连接池：

```
spring:
  redis:
    lettuce:
      pool:
        max-active: 128   # 从 32 扩到 128
        max-idle: 64
        min-idle: 16
        max-wait: 500ms
```

2. 消除群聊重复 Redis 调用：

把 `getGroupMemberIdsWithCache` 提前到 `sendMessage` 顶部调用一次，结果作为参数传入 `validateSenderAndReceiver`，彻底消除第二次调用：

```
// 修改前：validateSenderAndReceiver 内部和主流程各调一次，共 2 次 Redis 请求
// 修改后：主流程顶部调一次，结果透传，共 1 次 Redis 请求
List<Long> groupMemberIds = null;
if (request.getSessionType() == SessionType.GROUP.getValue()) {
    groupMemberIds = this.getGroupMemberIdsWithCache(sessionId); // 只调一次
}
validateSenderAndReceiver(request, sendUserId, senderUser, receiverUser, friend,
groupMemberIds);
```

3. mqSendExecutor 解耦 Tomcat 线程与 MQ 发送：

新建专用线程池 `MqSendExecutorConfig`（核心 16 线程，最大 64 线程，队列 8192，CallerRunsPolicy 兜底），把 `asyncSendOrderly` 的调用包进 `mqSendExecutor.execute()` 里。Tomcat 线程只负责提交任务，不等待 MQ 发送结果，立即返回响应给客户端：

```
// 修改前：asyncSendOrderly 直接在 Tomcat 线程调用，队列满时阻塞 Tomcat 线程 2000ms
rocketMQTemplate.asyncSendOrderly("IM_MSG_STORE", mqMessage, sessionId, callback,
2000L);

// 修改后：提交给专用线程池，Tomcat 线程立即返回
mqSendExecutor.execute(() ->
    rocketMQTemplate.asyncSendOrderly("IM_MSG_STORE", mqMessage, sessionId, callback,
```

```
2000L)
);
```

第三轮结果对比（直连 8084）：

| 指标 | 第二轮后（8084） | 第三轮后（8084） | |------|----------|----------| | 实际 RPS | 518 | 994 | | P50 | 61ms | 53ms | | P99 | 1957ms | 80ms | | P999 | 2134ms | 140ms | | timeout | 52 | 0 |

第三轮结果（经过 Gateway，端口 10010）：

| 指标 | 数值 | |------|------| | 实际 RPS | 989 | | P50 | 113ms | | P99 | 342ms | | P999 | 636ms | | timeout | 0 |

10010 的 P99 比 8084 高约 260ms，是 Gateway 层 JWT 验签 + HTTP 代理的正常开销，不是 Messaging 侧的问题。

五、三轮优化总结

| 轮次 | 发现的问题 | 定位方法 | 优化手段 | 结果（P99） | |------|----------|---------|---------|---------| | 第一轮 | `@Transactional` 持有 MySQL 连接时间过长，导致连接池耗尽 | jstack 调用栈抽样，发现线程卡在 HikariPool.getConnection | 去掉发送入口事务，落库交由 MQ 消费端异步完成 | 48s → 30s | | 第二轮 | `asyncSendOrderly` + `future.get` 构成假同步，阻塞 Tomcat 线程 3s | jstack 稳态快照，线程全卡在 CompletableFuture.get | 改为真异步回调，失败写 failover 表重投 | 30s → 950ms | | 第三轮 | Redis 池 32 个连接不够 61 个并发请求；群聊重复调 Redis；asyncSendOrderly 在 producer 队列满时仍阻塞 Tomcat | jstack + Little's Law 推算池容量；代码审查发现重复调用；P99 卡在 2000ms 与超时参数对应 | 扩 Redis 池到 128；合并重复调用；mqSendExecutor 解耦 | 950ms → 80ms |

最终成果： 在 2C8G 单节点、300 并发连接、目标 1000 RPS 下，实际吞吐 994 RPS，P50=53ms，P99=80ms，超时归零。每一轮优化的根因、定位手段和量化结果形成完整闭环。

一、Netty Q&A 还缺什么 简历 bullet 里提到了"单设备踢人"和"优雅停机"，但 Q&A 完全没覆盖。这两个是面试官顺着简历就能问到的点，必须补上。另外 Pipeline 的 Handler 链设计也是 Netty 必问细节。

补充 Q&A：

单设备踢人是怎么实现的？

A：在握手处理逻辑里，当新连接完成 JWT 鉴权、拿到 userId 之后，我会先去本地会话表里查这个 userId 是否已经有一个旧 Channel。如果有，说明同一个账号在当前节点上已经有活跃连接，我会向旧 Channel 推送一条"账号在其他地方登录"的通知，然后主动调用旧 Channel 的 close()，同时从本地会话表里删掉旧映射。之后再把新 Channel 注册进本地会话表，并向 Redis 写入新的全局在线路由。这样就做到了同一账号在同一节点上最多只有一个活跃连接。

优雅停机是怎么做的？

A：Netty 的优雅停机核心是关闭顺序：先停 Boss EventLoopGroup，让它不再接受新连接；再给 Worker EventLoopGroup 一个安静期，让它处理完 Pipeline 里还在途的事件；最后等 Business 线程池的任务队列排空。在停机前，我还会遍历本地会话表，向所有活跃连接推送"服务即将重启"的通知，提示客户端提前发起重连，而不是等 TCP 被动断开再重连。这样能把重连风暴的规模提前分散一部分。当前项目里这个流程是靠 JVM Shutdown Hook 触发的，调用 eventLoopGroup.shutdownGracefully(quietPeriod, timeout, TimeUnit.SECONDS) 来控制窗口期。

Pipeline 里的 Handler 链是怎么排列的？哪些在 Worker 线程，哪些提交到 Business 线程？

A：我的 Pipeline 大体分两段。前半段跑在 Worker 线程里，处理轻量的 IO 层工作：HttpServerCodec（HTTP 编解码）、HttpObjectAggregator（聚合 HTTP 请求体）、WebSocketServerProtocolHandler（处理握手升级和控制帧）、IdleStateHandler（空闲检测）。这些 Handler 本质上是协议解析和状态维护，执行快，不能阻塞。后半段是我自己写的 BusinessHandler，这里面包含 JWT 鉴权、用户身份绑定、在线路由注册、心跳处理和消息分发。这些操作要和 Redis 交互，不能留在 Worker 线程，所以 BusinessHandler 在 channelRead0 里把任务提交到 Business 线程池，Worker 线程立即返回继续处理下一个 IO 事件。

二、消息主链路完整复盘 一句话层 这块我做的是 IM 消息从客户端发出到落库、推送、兜底的全链路。核心目标有两个：一是设计保证消息不丢、不重、有序；二是发送接口本身足够轻，不被任何下游操作阻塞。我后来通过三轮 wrk2 压测把发送入口从 491 RPS/P99 48s 调到 994 RPS/P99 80ms，把每一轮优化都量化了出来。

方案层 为什么用 Lua 脚本把去重（SETNX）和 seq 生成（INCR）合并成一次原子操作，而不是分开做？

A：因为如果分开做，就会有竞态窗口。比如请求 A 先 SETNX 成功了，判定这条消息没有重复，然后去做 INCR 拿 seq；但在 SETNX 到 INCR 之间，如果请求 B 也带着同一个 clientMsgId 进来，SETNX 发现 key 已经存在，判定为重复直接返回，但这时 A 的 INCR 还没执行，B 就可能先拿到 seq。这样就会出现同一条消息被两个请求都处理的中间状态。用 Lua 脚本把两步合并成一次 Redis 原子执行，就不存在这个窗口：要么 SETNX 成功并立即拿到 seq，要么 SETNX 失败直接返回 -1 表示重复，整个判断和分配过程是原子的。

为什么 seq 用 Redis INCR 而不是数据库自增 ID 或者 Snowflake？

A：Snowflake 生成的是全局唯一 ID，但不是会话内连续递增的，客户端没办法用它判断消息是否有间隙、是否需要补拉。数据库自增 ID 可以做到递增，但把 seq 写库和消息发送绑定在同一个请求里，会拖慢发送入口，而且跨节点竞争自增也麻烦。Redis INCR 是针对 sessionId 维度做的，每个会话单独一个计数器，保证会话内消息 seq 连续递增，不同会话的 seq 互不影响。这样客户端收到消息后，只需要比较当前 seq 和本地记录的最大 seq，如果有跳跃就说明有消息漏了，按 sessionId + seq 区间去补拉就可以了。

为什么 MessageStoreListener 改成 CONCURRENTLY，而不是保持 ORDERLY？

A：ConsumeMode.ORDERLY 在 RocketMQ 里意味着同一个 messageQueue 上的消息必须串行消费，上一条没消费完，下一条不会开始。我的 MessageStoreListener 里有三步同步操作：MySQL INSERT、Redis multiGet 查路由、syncSendOrderly 转发，每条消息处理耗时在几十毫秒量级。在 1000 QPS 压力下，这三步串行叠加就会让 messageQueue 出现积压，消费速度远低于生产速度。

改成 CONCURRENTLY 之后，多条消息可以在不同线程上并发落库和转发，消费吞吐量线性提升。这里可以用并发的前提是：seq 已经在发送端由 Redis INCR 原子分配好了，消费端只负责落库和转发，不需要在消费时再保证顺序；而且 DuplicateKeyException 已经作为幂等兜底，重复消费的消息会被静默忽略。

为什么路由查询要放在 MessageStoreListener 里做，而不是发送接口里先查好路由再投递？

A：有两个原因。第一，在线路由是实时变化的，发送接口查到的路由到 MQ 消费时可能已经失效，放在消费端查能更接近推送时刻，减少脏路由的概率。第二，发送接口的职责应该尽量轻，只负责幂等判断、组包和投递，不应该去做跨服务的路由查询。把路由查询放在 MessageStoreListener 里，发送接口不感知接收方在哪个节点，全异步下沉，Tomcat 线程可以立即返回。

为什么要引入 mqSendExecutor 专用线程池，而不是直接在 Tomcat 线程里调 asyncSendOrderly？

A：asyncSendOrderly 虽然名字里有 async，但 RocketMQ producer 内部有一个 sendQueue，当 broker 写入压力大、队列满时，asyncSendOrderly 会阻塞调用线程等待队列空出位置，阻塞时长最多是传入的 timeout 参数，我这里是 2000ms。在直连 8084 端口、300 并发连接的压测下，我观察到 P99 刚好卡在约 2000ms，说明 Tomcat 线程频繁触发了这个阻塞。引入 mqSendExecutor 之后，Tomcat 线程只是把任务提交到这个线程池就返回，真正阻塞的是线程池里的工作线程，Tomcat 线程不被 MQ 发送路径拖住。

为什么离线消息要分 Redis ZSet（3天）和 MySQL（7天）两层，而不是只存一个地方？

A：只存 Redis 的话，ZSet 是内存结构，存储成本高，不适合长时间保存大量历史消息，而且 Redis 本身没有持久化消息的设计保证。只存 MySQL 的话，在线用户刚上线时拉最近几条消息，高频查询会打到数据库，延迟和压力都比较大。分两层的设计是：3 天以内的热数据存 Redis ZSet，按 userId 维度做 score=seq 的有序集合，上线时先查 Redis，快速拿到最近消息；超过 3 天的冷数据归档到 MySQL，低频访问走数据库。这样热路径走缓存，冷路径走持久层，平衡了性能和存储成本。

为什么补拉接口按 sessionId + seq 做增量，而不是按时间戳或者 messageId？

A：因为 seq 是会话内连续递增的整数，客户端只需要记录本地已经收到的最大 seq，补拉时传 beginSeq，服务端查 [beginSeq+1, endSeq] 区间就能精确返回缺失的消息，不会有遗漏也不会重复返回已有消息。时间戳有精度问题，同一毫秒内可能有多条消息，边界处理复杂。messageId 是无序的哈希值，不能直接用来做区间查询。seq 的连续性还有一个好处：如果服务端返回的 seq 区间里有跳跃（比如某条消息被删除），可以用 tombstone 占位填充，让客户端知道这个 seq 位置的消息是不存在的，而不是漏拉了。

failover 重投为什么要加 Redisson 分布式锁？

A：因为 failover 定时任务是从 im_msg_failover 表里捞待重投的消息，在多节点部署场景下，每个 Messaging 节点都跑着这个定时任务，如果不加锁，同一条 failover 消息可能被多个节点同时捞出来同时重投，造成重复发送。加了分布式锁之后，同一时刻只有一个节点能拿到锁并执行重投，其他节点等待或跳过。重投成功后把 failover 记录状态更新为已处理，下一轮不再重投。

细节层 Lua 脚本的具体逻辑是什么，返回值怎么判断？

A：脚本逻辑是：if redis.call('set', KEYS[1], '1', 'PX', ARGV[1], 'NX') then return redis.call('incr', KEYS[2]) end return -1。KEYS[1] 是去重 key（im:msg:dedup:{sessionId}:{clientMsgId}），KEYS[2] 是 seq 计数器（im:seq:{sessionId}），ARGV[1] 是 dedup key 的过期时间（毫秒）。脚本

返回正整数说明 SETNX 成功并拿到 seq，返回 -1 说明这个 clientMsgId 已经存在，判定为重复消息，直接返回幂等响应，不再进后续流程。

Cache-Aside 里随机 TTL 为什么是 24~48 小时，这个范围有什么依据？

A：随机范围本身不是精确计算出来的，而是一个工程经验值。核心目的是防止大量缓存同时到期（缓存雪崩）。如果所有用户信息、好友关系和群成员列表都设置固定 TTL，比如都是 24 小时，那么一批同时写入的缓存就会在同一时刻集中失效，造成那个时间点 Redis Miss 暴增，数据库被打。用 24 小时加一个 ThreadLocalRandom.nextInt(24) 的随机偏移，让不同 key 的过期时间分散在 24 到 48 小时之间，错开失效时间点，就能把雪崩风险摊平。具体范围根据业务更新频率决定：用户信息和好友关系变化不频繁，24~48 小时是合理的；群成员变化相对频繁一些，如果后续要优化可以单独收紧。

mqSendExecutor 的 CallerRunsPolicy 会不会让 Tomcat 线程被阻塞，从而让引入线程池失去意义？

A：CallerRunsPolicy 是队列满了之后的兜底策略，意思是当线程池的队列（8192 容量）满了、最大线程数（64）也跑满了，新来的任务会由调用方线程——也就是 Tomcat 线程——直接执行。这确实会让 Tomcat 线程临时承接 MQ 发送，造成短暂阻塞。但这是一个有意为之的权衡：放弃任务（AbortPolicy）会导致消息丢失；死等（不设置超时）会让调用方无限阻塞。CallerRunsPolicy 的语义是"当系统真的过载时，调用方降级兜底，给线程池喘息时间"，这比直接丢消息要安全得多。在正常压力下（1000 RPS），线程池不会满载，Tomcat 线程几乎不会触发 CallerRuns；只有在极端洪峰下才会触发，而那时整个系统本来就已经接近极限了。

MessageStoreListener 里的 syncSendOrderly 是同步转发，这会不会成为消费瓶颈？

A：会，这是当前实现的一个已知局限。syncSendOrderly 要等 broker 确认才返回，意味着消费线程在等 MQ 的网络往返。当前配了 consumeThreadNumber = 20，能并发处理 20 条消息，在目前的压测规模下够用。如果消息量继续上涨，更彻底的优化方向是把消费端的落库和转发也改成异步：消费线程只负责落库，然后发一个异步信号触发转发，或者把路由查询和转发拆到独立的 Consumer 去做。当前保持 syncSendOrderly 是因为它能让转发失败时 Consumer 直接重试，实现更简单；如果改成异步，需要自己处理转发失败的重试和补偿逻辑，复杂度上升。

三轮压测里，jstack 的分析方法论是什么？

A：jstack 分析我固定走三步。第一步看线程状态分布：统计 RUNNABLE / WAITING / TIMED_WAITING / BLOCKED 各有多少，判断是 CPU 打满型瓶颈还是等待型瓶颈。第一轮和第二轮大量线程是 WAITING/TIMED_WAITING，说明是阻塞等待，不是 CPU 计算不够。第二步看线程组分布：把线程名的末尾编号去掉，按组统计数量，找出数量最多的线程组。两轮都是 http-nio-8084-exec 占大头，说明 Tomcat 请求线程全被占住了。第三步抽样看调用栈：在 jstack 里手动找这些请求线程的 at 堆栈，看卡在哪一行。第一轮卡在 HikariPool.getConnection；第二轮卡在 CompletableFuture.get；第三轮卡在 Lettuce 连接池等待。三步走完就能准确定位是什么操作在阻塞请求线程，而不是凭感觉猜。

第三轮里用 Little's Law 推算 Redis 池不够，具体怎么算的？

A：Little's Law 是说：系统里同时在途的请求数 = 吞吐量 × 平均响应时间，即 $N = \lambda \times W$。我当时直连 8084，压测数据是 P50 = 61ms，目标 1000 RPS。代入公式：$N = 1000 \times 0.061 = 61$，也就是稳态下同时大约有 61 个请求在系统里。但 Redis Lettuce 连接池 max-active = 32，每个在途请求可能需要一个 Redis 连接，61 个并发请求只有 32 个连接可用，大概有 29 个请求在连接池门口排队

等连接。这就是尾延迟飙高的根本原因。把 max-active 扩到 128，同时在途的 61 个请求人均有富余的连接可用，尾延迟马上降下来了。

红包系统：发红包在 `@Transactional` 本地事务内完成余额原子扣减（MySQL 行锁 CAS，`UPDATE ... WHERE balance >= amount`）、创建红包记录与余额流水，事务提交后将金额列表预计算写入 Redis List（普通红包平均分配，拼手气红包二倍均值法），并通过 IM 消息通道下发红包消息；抢红包以前置 Redis count 拦截快速拒绝已抢完请求，再以 RocketMQ 事务消息保证 Redis 预扣与 MySQL 落库的最终一致——`executeLocalTransaction` 中执行四合一 Lua 原子脚本（SISMEMBER 防重 + LPOP 弹金额 + DECR 扣库存 + SADD 标记已领 + HSET 写 Pending 暂存），通过 `CompletableFuture.get` 将 Lua 结果同步回 HTTP 线程；消费端异步落库（行锁扣减剩余量 + 幂等校验 + 写领取记录 + 加余额 + 写流水），事务提交后清理 Pending Hash；Broker 回查时以 Pending Hash 是否存在精准决策 COMMIT/ROLLBACK；红包 24 小时到期时由延迟消息触发 CAS 状态更新（幂等）并退还剩余金额给发送方；经 wrk2 阶梯压测（10→50→150→300→500→800→1200 QPS），2C8G 单节点下 800 QPS 时 P99 = 51ms、零错误，1200 QPS 出现拐点（P999 从 89ms 飙至 804ms，瓶颈为事务半消息提交尾延迟），全程数据一致性验证零重复领取、零超发、领取总额分文不差。